

Revised EvidenceCodec Architecture for Fast Training and Immediate Deployment

Executive Summary

The revised system is a **raw-text evidence compressor** built around a **cross-encoder reranker**. It does not summarize the context, it does not emit latent vectors to the final LLM, and it does not use a teacher LLM in the deployment path. Instead, it chunks the original context into deterministic raw-text spans, retrieves a candidate pool with **hybrid sparse+dense+risk-aware recall**, reranks those spans with **ModernBERT-large** on the **joint** input pair `[CLS] query [SEP] chunk [SEP]`, and then selects the smallest set of raw-text chunks that fits a token budget while preserving answer-critical evidence. The model choice is well aligned with the available budget and hardware: ModernBERT is a modern encoder optimized for classification and retrieval workloads, trained on 2T tokens with native 8,192-token support, and the published large model has 28 layers and about 395M parameters. ¹

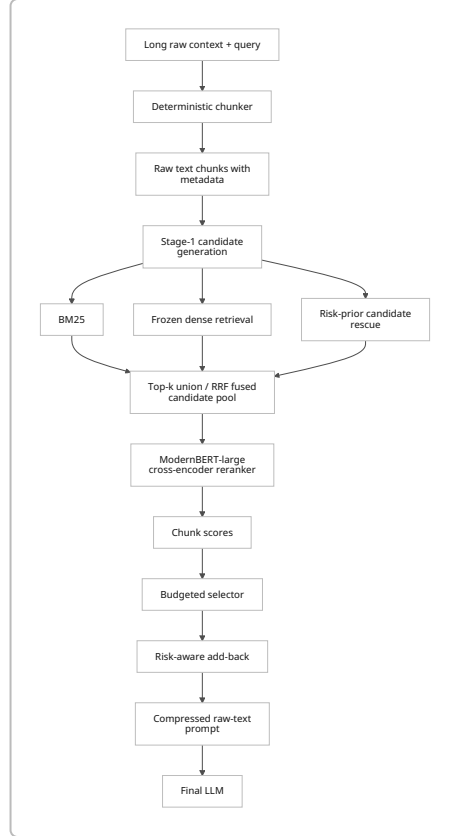
The best supervision sources are **HotpotQA** for large-scale sentence-level supporting facts and **QASPER** for document-grounded evidence over full NLP papers. HotpotQA provides about 113k Wikipedia QA pairs with sentence-level supporting facts, which makes it a natural source for chunk-level evidence labels. QASPER contains 5,049 questions over 1,585 NLP papers; evidence may span multiple paragraphs, tables, and figures, and 55.5% of its answerable questions require multi-paragraph evidence. Those properties make HotpotQA ideal for initial training and QASPER ideal for domain adaptation and demo-quality scientific-document examples. LongBench is the right benchmark for final evaluation because it standardizes long-context tasks into a common format and spans QA, summarization, few-shot, synthetic, and code tasks. ²

The main architectural change is that **atoms are no longer the primary training object**. The decisive object is the **evidence chunk**. “Atoms” still remain useful as an evaluation abstraction for silent-loss stress tests, but the production model is trained directly to score chunk usefulness for a query. This change simplifies the system, makes supervision easier to obtain from public datasets, and removes the need for runtime teachers or runtime verifiers. The verifier becomes an **offline evaluation oracle**, not an online component. ³

With **8 hours and 10 B200 GPUs**, the highest-value strategy is not to spend all capacity on a tiny student model. The strongest plan is to train a **ModernBERT-large cross-encoder** as the primary quality model, then distill its scores into a cheap student such as **LightGBM** or a small MLP over sparse, dense, and risk features. The production story then has two clean modes: a **quality mode** using the reranker directly and a **cheap mode** using the student. That split is practical because cross-encoders are accurate for pairwise scoring, while student models are cheap enough for latency-sensitive deployment. ⁴

Architecture Overview

The architecture below is the recommended implementation target.



The governing principle is simple: the final LLM should receive **raw text selected from the original context**, not an abstractive summary and not a latent representation. That keeps the system compatible with any downstream LLM API and preserves direct inspectability of what was retained. The chunker is therefore only a **segmentation stage**, the candidate generator is only a **high-recall filter**, and the reranker is the first component that should “understand” the query–evidence interaction. This division of labor is important because it explains why deterministic chunking is acceptable and why semantic understanding belongs in the reranker, not in the chunker. ⁵

The core scoring equation

For a query q and chunk x_i , the reranker score is

$$h_i = \text{Enc}_\theta([\text{CLS}] q [\text{SEP}] x_i [\text{SEP}])_{\text{CLS}}, \quad s_i = W h_i + b.$$

Here, Enc_θ is the ModernBERT-large encoder, h_i is the final hidden state at the `[CLS]` position, and s_i is a scalar usefulness logit. The notation Enc_{CLS} or a subscripted “CLS” at the output means **select the hidden state corresponding to the CLS token from the encoder output**. It is not a summary of the query alone and not a summary of the chunk alone; it is a **joint representation of the whole pair**, because BERT-style sequence-pair inputs are packed jointly as `[CLS] A [SEP] B [SEP]` and the final hidden state of `[CLS]` is used as the aggregate representation for classification. The BERT paper states this explicitly, and the Hugging Face model documentation explains that the sequence-pair format uses `[CLS] A [SEP] B [SEP]`. ⁶

For manual implementations, the safest choice is to use the **final hidden state at position 0**, i.e. `last_hidden_state[:, 0, :]`, rather than relying on a model-specific pooled output. Hugging Face’s BERT documentation notes that `pooler_output` is the first-token hidden state after an extra linear+tanh layer tied to auxiliary pretraining conventions. That is acceptable for legacy BERT, but for a ModernBERT reranker the cleanest and most portable approach is to put the classification head directly on the last-layer CLS state, or simply use `AutoModelForSequenceClassification`. ⁷

Quality mode and cheap mode

A strong system should expose two deployment modes.

Mode	Retrieval	Main scorer	Deployment cost	Use case
Quality mode	Hybrid sparse + dense + risk	ModernBERT-large cross-encoder	Medium	Demos, batch jobs, premium flows
Cheap mode	Hybrid sparse + dense + risk	Distilled student	Low	Default serving
Fallback mode	BM25 + risk	No learned reranker	Very low	Safety fallback

This split preserves the best model you can train tonight while still yielding a persuasive production story. Cross-encoders process text pairs jointly and are typically more accurate than embedding-only first-stage models for reranking because they evaluate interactions token-by-token instead of comparing two independently produced embeddings. Sentence Transformers’ documentation describes this distinction directly: a CrossEncoder jointly processes both texts and outputs a score, rather than producing reusable sentence embeddings. ⁸

Chunking and Candidate Generation

Why a deterministic chunker is acceptable

A deterministic chunker is acceptable **because chunking is not where semantic relevance is decided**. Its job is to manufacture candidate units that are coherent enough for the reranker to score. If chunking has high evidence recall and avoids catastrophic boundary mistakes, then semantic discrimination can be delegated to the cross-encoder. The reason this works is that the evidence annotations in the training data are already sentence- and paragraph-grounded: HotpotQA gives sentence-level supporting facts, while QASPER exposes paragraph- and sentence-level evidence fields over full papers. The chunker therefore needs to preserve those local evidence units, not solve the whole retrieval problem on its own. ⁹

That said, deterministic chunking absolutely has limits. It does not “understand” long-distance semantic dependencies, and it can split away essential qualifiers like negations, exceptions, or table-row context. QASPER is especially instructive here because evidence is often spread across multiple paragraphs and may involve tables or figures, which means that crude fixed-window token splitting will underperform section-aware or paragraph-aware segmentation. ¹⁰

Recommended chunker

The recommended chunker is **hierarchical, deterministic, and structure-preserving**.

1. Parse document structure whenever available: headings, subheadings, paragraphs, bullet items, code blocks, table rows, and figure captions.
2. Use the **final LLM tokenizer** for budget accounting, not the encoder tokenizer. Budget selection should be done in the token space that actually matters for the final prompt.
3. Use the **encoder tokenizer** only when packing `(query, chunk)` pairs for the cross-encoder.
4. Build chunks from contiguous structure units up to a target budget, then fall back to sentence splitting, then to token windows only if necessary.
5. Add **small overlap** between adjacent chunks from the same structural parent.
6. Carry **breadcrumbs** into chunk metadata, such as `document title > section > subsection`, and optionally prepend a short heading prefix to the chunk text fed into the reranker.

A practical default is:

Parameter	Recommended value
Target chunk size	160 tokens
Soft upper bound	192 tokens
Hard upper bound	224 tokens
Overlap	32 tokens
Query max length for reranker	64–96 tokens
Packed pair length	512 tokens
Preserve verbatim	Table rows, list items, code blocks, equations, captions
Never split around if avoidable	<code>not</code> , <code>unless</code> , <code>except</code> , <code>only if</code> , units, prices, dates

Those values are not arbitrary. They are a compromise between (i) giving the cross-encoder enough local context to judge usefulness and (ii) keeping enough candidates available under the stage-one cap. Because ModernBERT supports long contexts efficiently, there is no architectural need to make chunks tiny; but smaller chunks are easier to label from sentence-level supporting facts and easier to fit into a budgeted extraction system. ModernBERT’s native support for long inputs gives headroom, not an excuse to abandon disciplined chunk design. ¹

Chunking algorithm

A concrete deterministic algorithm is:

$$\text{Chunk}(D) = \text{Pack}(\text{Units}(D), L_{\text{target}}, L_{\text{max}}, o),$$

where `Units(D)` extracts structure-aware units, `Pack` greedily groups adjacent units until the target length is reached, L_{target} is the target chunk size, L_{max} is the hard maximum, and o is the overlap. If a unit exceeds L_{max} , recursively split:

$$\text{split_unit}(u) = \begin{cases} \text{paragraphs}(u), & \text{if structural children exist} \\ \text{sentences}(u), & \text{otherwise if sentence splitter succeeds} \\ \text{token_windows}(u), & \text{fallback} \end{cases}$$

This is deterministic, easy to debug, and easy to project gold evidence onto. The crucial point is that “understanding semantics” is not the burden of this step. The burden of this step is **preserving local coherence and maximizing downstream evidence recall**.

How to mitigate semantic-splitting failures

The chunker’s failure mode is not low precision; it is **silent corruption** from broken boundaries. The fixes should therefore be conservative:

- **Rule-based don’t-split markers** for negation and exception phrases.
- **Adjacency rescue**: if a selected chunk begins or ends with a connective or incomplete sentence, allow the neighbor to be added at reduced threshold.
- **Heading breadcrumbs**: section titles often restore semantic meaning that a local window alone would miss.
- **Overlap**: small but nonzero overlap prevents one-token boundary catastrophes.
- **Structure preservation**: never split a table row, code block, or numbered requirement unless absolutely forced.
- **Evaluation at the candidate recall stage**: if gold evidence is regularly missing from the candidate set, the chunker is wrong even if the reranker looks good.

These mitigations are especially important for datasets like QASPER and LongBench, where evidence may span multiple paragraphs and where long input tasks require reasoning over structured or semi-structured content. ¹¹

Why BM25 alone is not enough

BM25 remains an excellent sparse baseline and should absolutely be in the system. The IR book’s treatment of bag-of-words models notes that such models ignore token order and retain counts rather than true semantic structure; its BM25 chapter further motivates BM25 as a term-frequency and document-length adjusted ranking scheme for full-text collections. That is precisely why BM25 is useful here: it is cheap, lexical, and good at exact term matching. But it is also why BM25-only retrieval is insufficient for long-context evidence compression. It is weak on paraphrase, synonymy, and semantically related evidence that does not share many query terms. ¹²

Dense retrieval exists to cover that semantic gap. DPR showed that learned dense retrieval can outperform strong BM25 baselines on top- k passage retrieval accuracy in open-domain QA, and Contriever later showed strong unsupervised recall improvements over BM25 on most BEIR datasets for Recall@100. The engineering implication is straightforward: **sparse and dense retrieval fail differently**, so stage-one recall is better when both are used. ¹³

Recommended candidate generator

The safest eight-hour implementation is not a monolithic hybrid “score” from the start. It is a **union-of-top-lists generator** with optional RRF tie-breaking:

1. $C_{\text{bm25}} = \text{Top}_{96}(\text{BM25})$
2. $C_{\text{dense}} = \text{Top}_{96}(\cos(e_q, e_i))$
3. $C_{\text{risk}} = \text{Top}_{32}(\text{risk-priority})$
4. Candidate pool $C = \text{Unique}(C_{\text{bm25}} \cup C_{\text{dense}} \cup C_{\text{risk}})$
5. If $|C| > 256$, keep the top 256 by Reciprocal Rank Fusion or normalized score sum.

A standard RRF tie-break looks like

$$\text{RRF}(i) = \sum_{m \in \{\text{bm25}, \text{dense}, \text{risk}\}} \frac{\alpha_m}{c + \text{rank}_m(i)},$$

with $c = 60$ and $\alpha_{\text{bm25}} = \alpha_{\text{dense}} = 1$, $\alpha_{\text{risk}} \in [0.5, 1.0]$ as a starting point. The value of this design is practical: BM25 and cosine scores live on different scales, while rank fusion is robust and quick to tune. This is a recommendation rather than a dataset-imposed truth, but it is a good fit for the constraints and is consistent with the general literature on multi-stage retrieval. DPR also gives a strong practical precedent for pairing dense retrieval with BM25-derived hard negatives rather than treating BM25 as obsolete. ¹⁴

Risk-prior design

The risk prior is not a second semantic model. It is a **cheap, query-aware penalty against dropping fragile evidence**.

Define binary or count features f_{ij} for chunk x_i :

- f_{neg} : contains negation (not, no, never)
- f_{exc} : contains exception (unless, except, only if, provided that)
- f_{num} : contains numbers or quantities
- f_{date} : contains dates or years
- f_{money} : contains currency, percentages, or prices
- f_{cmp} : comparison markers (greater, less, before, after, higher, minimum)
- f_{code} : code identifiers, file paths, function names, config keys
- f_{table} : table row, caption, or figure/table evidence
- f_{eq} : equation or formula patterns
- f_{entity} : named-entity overlap with query
- f_{heading} : heading overlap or section-title overlap with query
- f_{numov} : numeric overlap with query if the query contains numbers

A practical **hand-initialized risk score** is

$$r_i = 2.0f_{\text{neg}} + 2.0f_{\text{exc}} + 1.5f_{\text{num}} + 1.5f_{\text{date}} + 1.5f_{\text{money}} + 1.2f_{\text{cmp}} + 1.5f_{\text{code}} + 1.5f_{\text{table}} + 1.2f_{\text{eq}} + 1.0f_{\text{entity}} + 0.5f_{\text{head}}$$

Then normalize within-query:

$$\hat{r}_i = \frac{r_i - \min_j r_j}{\max_j r_j - \min_j r_j + \varepsilon}.$$

The reason numbers, comparators, and tables matter is visible in datasets like DROP and QASPER. DROP is explicitly constructed to require discrete reasoning over paragraph content, including addition, counting, and sorting, which means deleting the wrong number often makes the answer silently wrong. QASPER frequently requires multi-paragraph evidence and sometimes table/figure evidence, which creates the same fragility under compression. ¹⁵

How the risk prior is used

Use the risk prior in **two places**, not everywhere.

First, use it lightly in stage-one candidate rescue so dangerous chunks are not fully excluded from the reranker pool. Second, use it strongly in **post-rerank add-back**. Let

$$p_i = \sigma(s_i)$$

be the reranker probability. Define add-back priority

$$a_i = \mathbf{1}[p_i \geq \tau_{\text{rel}}] \cdot \frac{\hat{r}_i}{\sqrt{\text{tokens}(x_i)}}.$$

A chunk is eligible for rescue if it is at least moderately relevant and materially risky. A practical default is

$$\tau_{\text{rel}} = 0.35, \quad \tau_{\text{risk}} = 0.55, \quad B_{\text{addback}} = 0.10 B_{\text{total}}.$$

Then sort dropped chunks by a_i and add them back until either the add-back budget is exhausted or $\hat{r}_i < \tau_{\text{risk}}$. This is intentionally cheap and conservative. It is not an LLM verifier. It is a structured hedge against silent-loss failure cases.

How risk weights should be calibrated

If there is time, do not leave the weights purely heuristic. Calibrate them against a **silent-loss dev suite**:

1. Take held-out HotpotQA and QASPER examples.
2. Build compressed prompts under fixed budgets.
3. Remove one candidate chunk at a time from the kept set.
4. Label the removal as harmful if gold evidence recall drops or if downstream answer correctness falls relative to full-context answering.
5. Fit a tiny logistic regressor on the risk features f_i to predict harmful omission.

That gives

$$\Pr(\text{silent-loss} \mid f_i) = \sigma(w^\top f_i + c).$$

Then either use the learned probability directly as $\hat{r}_i$, or only use it to refine the manual weights. Under an 8-hour deadline, the recommended order is: **manual initialization first, grid-search thresholds second, learned calibration only if the main reranker is already stable**.

Cross-Encoder, Labels, and Losses

What the training example really is

The supervised unit is **not a full document** and not an “atom.” It is a **query-chunk pair**:

$$(q, x_i, y_i).$$

The label y_i means “this chunk should be kept because it contains or materially contributes to the gold evidence.” That is a supervised setup. BCE is appropriate because after preprocessing, each chunk becomes a labeled training instance. The fact that chunks are created by preprocessing rather than supplied directly by the dataset is not a problem; the label is induced by **projecting evidence annotations onto chunk boundaries**.

How y_i is defined when chunks are arbitrary

This is the key preprocessing step.

For **HotpotQA**, the dataset card shows that each example has a question, an answer, a `context` made of titles plus sentence lists, and `supporting_facts` with titles and sentence IDs. That means if the chunker groups contiguous sentences into chunk x_i , then $y_i = 1$ if x_i contains any supporting-fact sentence, and $y_i = 0$ otherwise. Because HotpotQA supervision is sentence-level, label projection is exact enough for an eight-hour system. ¹⁶

For **QASPER**, the dataset card and paper show two relevant evidence fields: `highlighted_evidence`, which is sentence-level, and `evidence`, which maps the evidence to whole paragraphs or figure/table units. If your chunker preserves paragraphs whenever possible, a good positive-label rule is

$$y_i = \begin{cases} 1, & \text{if chunk } x_i \text{ overlaps highlighted evidence or evidence paragraphs} \\ 0, & \text{otherwise.} \end{cases}$$

If you want softer supervision, use an overlap-proportion label such as

$$y_i = \min\left(1, \frac{\text{\#evidence tokens in } x_i}{\text{\#tokens in } x_i}\right).$$

In practice, plain binary labels are usually sufficient for the first run. The important conceptual point is that the chunk is arbitrary **only before** preprocessing; after label projection, it is a standard supervised chunk instance. ¹⁷

Dataset shape and practical chunk sizes

The training data looks like this after preprocessing.

Dataset	Query shape	Context shape	Gold evidence granularity	Recommended chunk unit
HotpotQA	Short natural question	Multi-paragraph Wikipedia sentences grouped by article title	Sentence-level supporting facts	Sentence packs
QASPER	Information-seeking paper question	Full paper text with sections, paragraphs, tables, figures	Sentence/paragraph/table/figure evidence	Paragraph-first chunks
LongBench	Short input/command field	Long context string	Usually answers only; task-dependent	Eval only, same chunker

HotpotQA’s dataset card exposes the exact structural fields used for label projection and shows training splits around 90k examples in the main training partition. QASPER’s dataset card exposes `full_text`, `qas`, and paragraph/sentence evidence fields, with train/validation/test splits at 888/281/416 papers. LongBench standardizes examples into `input`, `context`, `answers`, `length`, and metadata, and its task statistics show that average English lengths vary widely by task, including about 9,151 for HotpotQA-derived LongBench tasks and 3,619 for Qasper-derived LongBench tasks. ¹⁸

For implementation, the model should not be trained on raw papers of arbitrary length. It should be trained on chunks of roughly **160–224 final-LLM tokens**, which usually fit comfortably inside a **512-token query+chunk pair** for the reranker.

Why BCE is not enough by itself

BCE teaches the model absolute chunk usefulness, which is good. But retrieval and compression are fundamentally **ordering tasks**: the important error is not merely that a negative chunk gets a high probability, but that a negative chunk gets ranked above a positive one **under the same query**. Pairwise ranking loss addresses that directly.

Use the combined loss

$$\mathcal{L} = \lambda_{\text{bce}} \cdot \text{BCEWithLogits}(s_i, y_i) + \lambda_{\text{rank}} \cdot \frac{1}{|\mathcal{P}|} \sum_{(p,n) \in \mathcal{P}} \max(0, m - s_p + s_n),$$

where p is a positive chunk for the same query, n is a sampled negative chunk, m is the ranking margin, and $\mathcal{P}$ is the set of sampled positive–negative pairs.

A good first setting is

$$\lambda_{\text{bce}} = 1.0, \quad \lambda_{\text{rank}} = 0.5, \quad m = 0.2.$$

BCE stabilizes learning because it is a direct chunk-level supervision signal. Pairwise loss sharpens the ranking because it explicitly punishes misordering. That matters most for **hard negatives**, where the question terms or topic overlap heavily but the chunk is not actual evidence. DPR’s analysis of hard

negatives and in-batch negatives is a strong practical justification for this design: the paper shows that in-batch negatives help substantially, and adding a single BM25 hard negative further improves top- k retrieval accuracy. ¹⁹

Hard-negative mining

Hard negatives are necessary if you want the reranker to learn the difference between “looks relevant” and “is evidence.”

Use three negative types:

1. **Random negatives**: arbitrary chunks from the same document or batch. These make the task easy and stabilize early training.
2. **BM25 hard negatives**: top lexical matches that do **not** overlap gold evidence.
3. **In-batch negatives**: positive chunks from other queries in the same batch, treated as negatives for the current query.

DPR is again instructive here. It explicitly compares random negatives, BM25 negatives, and in-batch negatives, and reports clear gains from in-batch reuse and from the addition of BM25 hard negatives. The practical lesson is that hard negatives are not optional garnish; they materially change ranking quality. ²⁰

A concrete implementation is:

- For each positive chunk, sample **4 BM25 hard negatives** from the same example.
- Sample **4 random negatives** from the same example or corpus.
- Use **all other positives in the batch** as in-batch negatives.
- Refresh the BM25 hard-negative cache once after the first hour of training if time permits, using the newest model scores for semi-online mining.

If time is tight, do not implement a fancy online mining loop. Start with **offline BM25 hard negatives plus in-batch negatives**. That gives most of the benefit with almost none of the engineering risk.

All trainable parameters

The trainable parts are limited and should stay limited.

Component	Trainable tonight	Notes
ModernBERT-large encoder θ	Yes	Main parameter mass, about 395M params in the model card
Classification head W, b	Yes	Scalar usefulness logit
Student model ψ	Yes	LightGBM or small MLP on engineered features
Temperature / calibration scalar T	Optional	Useful if you need better confidence calibration

Component	Trainable tonight	Notes
Learned risk calibrator w_r	Optional	Only if risk calibration time remains
Dense embedding model	No, recommended frozen	Keep first-stage dense leg simple tonight
BM25 index	No	Nonparametric
FAISS index	No	Nonparametric search structure
Chunker rules	No	Tuned heuristics, not learned

This is one of the strongest aspects of the design: most of the pipeline is deterministic or frozen, so the main optimization budget is concentrated in the model that matters most.

Datasets and Supervision Strategy

Training datasets

The training plan should prioritize evidence supervision, not general long-context generation.

HotpotQA should be the primary pretraining dataset because it is large and exposes sentence-level supporting facts. The paper and dataset card both emphasize sentence-level supporting facts, which map directly to positive chunk labels. The main training split is about 90k examples in the standard configuration. ²¹

QASPER should be the second-stage fine-tuning dataset because it is much closer to the “compress long documents for QA” story. It contains 5,049 questions over 1,585 papers, with paragraph-/sentence-/table-/figure-level evidence. The official paper highlights that evidence may span multiple paragraphs and that tables and figures are sometimes necessary. The dataset card exposes the exact evidence fields needed for label projection. ²²

LongBench should be used **for evaluation, not for primary training**. Its value is breadth and long-context realism, not chunk-level evidence labels. The dataset card standardizes inputs into short `input` fields and long `context` fields, and the task statistics reveal substantial variation in average length across tasks. This makes it ideal for comparing compression strategies under a shared evaluation harness. ²³

Offline evaluation meaning for $k = 256$

When the report refers to **offline evaluation at $k = 256$** , it means the following procedure.

For each held-out example:

1. Chunk the context deterministically.
2. Use stage-one retrieval to produce a candidate pool of at most 256 chunks.
3. Do **not** call the final LLM yet.
4. Measure whether the gold evidence chunks are present in the candidate pool.

This isolates the candidate generator. It answers the question: *before reranking, did stage one even keep the right evidence in play?*

The critical offline metrics are:

$$\text{EvidenceRecall@}k = \frac{\text{\#gold evidence chunks retrieved in top } k}{\text{\#gold evidence chunks}},$$

and a stricter example-level metric:

$$\text{AllEvidenceCovered@}k = \mathbf{1}[\text{every gold evidence chunk is in the top-}k].$$

For budget-aware evaluation, also compute an **oracle upper bound** inside the candidate pool. Let B be the token budget and let each chunk have cost c_i . Using gold evidence labels, solve or approximate a knapsack over the candidate pool to maximize evidence recall under budget:

$$\max_{z_i \in \{0,1\}} \sum_i y_i z_i \quad \text{s.t.} \quad \sum_i c_i z_i \leq B.$$

This “oracle@budget within $k = 256$ ” tells you whether the failure is due to stage-one candidate recall or due to stage-two reranking/selection. That distinction is extremely useful under a compressed timeline.

Recommended evaluation metrics

Use four layers of evaluation.

Candidate-stage metrics

- EvidenceRecall@64/128/256
- AllEvidenceCovered@64/128/256
- OracleRecall@Budget within candidate pool

Reranker-stage metrics

- MRR / nDCG over labeled chunks
- Positive-over-hard-negative accuracy
- Top- n evidence hit rate

Compression-stage metrics

- EvidenceRecall@Budget
- Compression ratio
- Average kept tokens

End-to-end metrics

- Answer F1 / EM where benchmark supports it
- Silent-loss rate
- Latency: chunking, candidate generation, reranking, selection

Define **silent-loss rate** as

$$\text{SLR} = \frac{1}{N} \sum_{j=1}^N \mathbf{1} [\text{FullContextCorrect}_j = 1 \wedge \text{CompressedCorrect}_j = 0].$$

This is the right metric for a compression system because it isolates the cases where compression broke an answer that the full-context pipeline would otherwise have gotten right.

Eight-Hour Training and Evaluation Plan

Recommended training schedule

The schedule below assumes that the expensive part is the cross-encoder and that all supporting infrastructure is kept as simple as possible.

Time	Goal	Main outputs
Hour one	Data pipeline + chunk label projection	HotpotQA and QASPER chunk datasets
Hour two	Sparse/dense candidate baseline	BM25, frozen dense, risk flags, offline candidate eval
Hours three to four	Main reranker training	ModernBERT-large checkpoint
Hour five	Reranker integration	End-to-end budgeted selector
Hour six	Student distillation	LightGBM or small MLP student
Hour seven	Offline evaluation + ablations	k , chunk size, loss ablations
Hour eight	Demo curation + final tables	Side-by-side examples and benchmark summary

Ten-GPU parallelization plan

A 395M-parameter encoder is small enough that you should **not** waste all ten B200s on a single distributed job. Use the hardware for parallel insurance and evaluation.

GPU set	Job
GPUs 1–4	Main ModernBERT-large training run
GPUs 5–6	Baseline/ablation run: ModernBERT-base or different loss mix
GPU 7	Candidate generation + dense embedding prep
GPU 8	Teacher-score generation for distillation

GPU set	Job
GPU 9	Student training + risk-threshold calibration
GPU 10	Evaluation notebooks + demo server + spare/debug

This is more robust than a single heroic training run. Cross-encoder fine-tuning is usually stable, but data and labeling bugs are not. Parallelizing across ablations is the better use of the cluster.

Recommended reranker hyperparameters

Hyperparameter	Recommended value
Model	<code>answerdotai/ModernBERT-large</code>
Sequence length	512
Query truncation	64-96 tokens
Chunk truncation	Remainder of 512
Precision	bf16
Optimizer	AdamW
Learning rate	1e-5 to 2e-5
Weight decay	0.01
Warmup	6%
Epochs	1-2
Global batch size	128-256 effective
Loss	BCE + pairwise margin ranking
Margin m	0.2
Hard negatives per positive	4
Random negatives per positive	4
Checkpoint selection	Best dev EvidenceRecall@Budget or reranker nDCG

These are conservative fine-tuning settings for a BERT-style encoder. They are meant to converge quickly and reliably; they are not intended as a final exhaustive sweep.

Recommended student distillation

The student should imitate the cross-encoder's logit or probability:

$$\hat{s}_i^{(\text{student})} \approx s_i^{(\text{teacher})}.$$

Train it on features that are already available cheaply:

- BM25 score
- Dense cosine similarity
- Query term overlap
- Entity overlap
- Numeric overlap
- Heading overlap
- Position in document
- Token count
- Redundancy score
- Risk features
- Neighbor-selected flags or local-context flags

For speed tonight, **LightGBM** is often the best student because it trains fast and handles mixed sparse/dense/tabular signals with little tuning. A two-layer MLP is also fine if you want a single PyTorch stack, but LightGBM is the safest choice for time-to-first-result.

Experiments worth running tonight

The indispensable ablations are these:

Experiment	Why it matters
Truncation baseline	Shows naive failure
BM25-only candidate generation	Tests lexical recall floor
Dense-only candidate generation	Tests semantic recall floor
Hybrid candidate generation	Verifies stage-one improvement
Hybrid + reranker	Main quality result
Hybrid + reranker + risk add-back	Demonstrates silent-loss reduction
Distilled student	Production story
$k \in \{64, 128, 256, 512\}$	Explains candidate-recall tradeoff
Chunk size $\in \{128, 160, 224\}$	Explains segmentation tradeoff
BCE-only vs BCE+ranking	Shows ranking loss value

If time collapses, prioritize the first six.

Implementation Details and Final Recommendations

Exact recommendation for the encoder and pooling

Use **ModernBERT-large** as a sequence-classification-style cross-encoder. Feed the pair jointly, and place the scoring head on the **last-layer CLS hidden state**. In notation:

$$h_i = H_0^{(L)}, \quad s_i = Wh_i + b.$$

The CLS vector here is the **joint pair representation**. It is not “a summary of q ” and not “a summary of x_i ” in isolation. The subscripted “CLS” at the end of an encoder expression simply means “take the representation corresponding to the CLS position from the final encoder output.” This usage follows the original BERT paper’s formulation, where the first token is always [CLS] and its final hidden state is used as the aggregate representation for classification. ²⁴

Exact recommendation for stage one

If you need one decisive sentence: **do not use BM25 alone**. Use BM25 + frozen dense retrieval + risk rescue, cap at 256 unique candidates, and rerank. BM25 is cheap and exact; dense retrieval covers paraphrase and semantic recall; risk rescue preserves fragile spans that are commonly lost in compression. The sources jointly support each leg: BM25 is the classical sparse backbone, dense retrieval has repeatedly shown recall gains over sparse baselines in QA retrieval, and tasks like DROP and QASPER provide direct evidence that numbers, tables, and multi-paragraph support matter. ²⁵

Exact recommendation for the chunker

Use the deterministic hierarchical chunker. Do **not** attempt a learned semantic chunker tonight. A learned chunker adds engineering uncertainty without changing the core logic of the system. Deterministic structure-aware chunking is sufficient if you aggressively evaluate candidate evidence recall and if you include overlaps, no-split rules, and adjacency rescue. The reranker is where semantics belongs.

Exact recommendation for training data

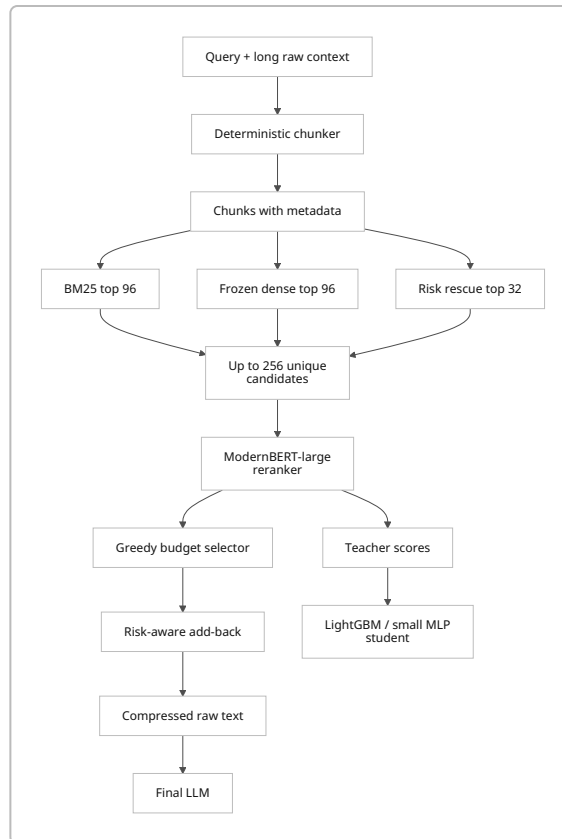
Train in this order:

1. **HotpotQA** pretraining for large supervised signal
2. **QASPER** adaptation for document-grounded evidence selection
3. **LongBench** evaluation only
4. Optional **DROP** stress tests for number-sensitive silent-loss behavior

That is the cleanest allocation of the remaining time. HotpotQA gives scale and exact sentence-level supervision. QASPER gives realism for long scientific documents. LongBench gives credible long-context evaluation. DROP gives a sharp story for why the risk prior exists. ²⁶

Final architecture summary

The final recommended system is:



The most important conceptual clarifications are these:

- The system compresses **raw text**, not latent vectors.
- The chunker is deterministic and structure-aware; it is not the semantic brain.
- BM25 alone is not enough because stage-one recall must cover both lexical and semantic evidence.
- The risk prior is a cheap hedge against silent corruption from dropped numbers, negations, exceptions, tables, and code.
- Offline $k = 256$ evaluation means evaluating candidate coverage on a held-out set **before** the final LLM, so you can isolate stage-one recall from stage-two ranking and stage-three compression.
- y_i is a chunk-level supervised label induced by projecting supporting-fact or evidence annotations onto deterministic chunks.
- BCE is valid because the training unit is the chunk after preprocessing; pairwise ranking loss is added because the actual task is ranking, not only classification.
- The trainable center of the system is the cross-encoder; everything else should remain as cheap and stable as possible.

That architecture is rigorous enough to implement immediately, strong enough to justify the hardware budget, and clean enough to demo persuasively in eight hours.

1 [2412.13663] Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference

<https://arxiv.org/abs/2412.13663>

2 3 21 26 [1809.09600] HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering
<https://arxiv.org/abs/1809.09600>

4 answerdotai/ModernBERT-base · Hugging Face
<https://huggingface.co/answerdotai/ModernBERT-base>

5 8 CrossEncoder — Sentence Transformers documentation
https://sbert.net/docs/package_reference/cross_encoder/model.html

6 24 arxiv.org
<https://arxiv.org/pdf/1810.04805>

7 BERT · Hugging Face
https://huggingface.co/docs/transformers/en/model_doc/bert

9 16 18 hotpotqa/hotpot_qa · Datasets at Hugging Face
https://huggingface.co/datasets/hotpotqa/hotpot_qa

10 11 22 arxiv.org
<https://arxiv.org/pdf/2105.03011>

12 25 Introduction to Information Retrieval
<https://nlp.stanford.edu/IR-book/pdf/irbookonlinereading.pdf>

13 Dense Passage Retrieval for Open-Domain Question Answering
https://arxiv.org/abs/2004.04906?utm_source=chatgpt.com

14 19 20 arxiv.org
<https://arxiv.org/pdf/2004.04906>

15 DROP: A Reading Comprehension Benchmark Requiring Discrete Reasoning Over Paragraphs
https://arxiv.org/abs/1903.00161?utm_source=chatgpt.com

17 allenai/qasper · Datasets at Hugging Face
<https://huggingface.co/datasets/allenai/qasper>

23 zai-org/LongBench · Datasets at Hugging Face
<https://huggingface.co/datasets/zai-org/LongBench>